

Exercicio 1: Problemas con aplicacións non stateless na escalabilidade

Explicación Vamos crear un novo proxecto de Git utilizando xa unha aplicación implantada preparada para produción. Esta para funcionar utiliza sesións e ven con posibilidade de almacenar as sesións en ficheiros ou nun servidor Redis.

Imos facer unha primeira proba na que despregamos a aplicación gardando a sesión en ficheiros.

1. Crea un novo proxecto en Gitlab de nome `T05.04 Nginx:Balanceo de carga`

. Clónao posteriormente no teu equipo.

No teu equipo, crea un ficheiro `docker-compose.yaml`

para despregar a seguinte imaxe:

- Nome imaxe: `mvarelasanclemente/balanceosessions`

Nome do contedor: `balanceo01`

Non necesita volumes.

Mapea o porto 80 no 9003

Hai estas 3 variables de sesión:

- `APP_NAME`

: Nome da aplicación. Neste caso pon `iniciais_01`

.

`SESSION_DRIVER`

: O seu valor pode ser `files` se imos utilizar ficheiros para almacenar a sesión ou `redis` se a imos almacenar nos servidor Redis. De momento utiliza `files`.

`REDIS_URL`

- - : O seu valor é a URL do servidor REDIS que se usará se utilizamos para almacenar as sesións.

Executa o `docker compose up`

e comproba que podes acceder a URL: `localhost:9003`. Actualiza varias veces para ver que o contador de veces que accediches a URL vai aumentando. **Entrega capturas** nas que se vexa como aumenta o contador e o ficheiro `docker-compose.yaml`

Benvido!

Aplicación cvd_01

Accediches a esta páxina 4 veces nesta sesión.

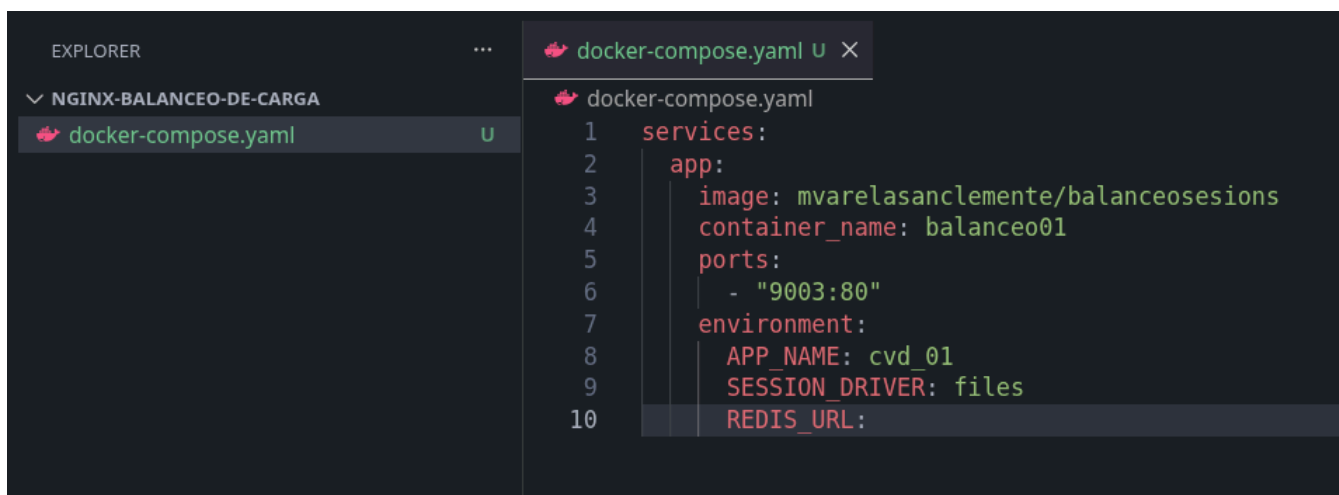
Método de sesión: **files**

Benvido!

Aplicación cvd_01

Accediches a esta páxina 12 veces nesta sesión.

Método de sesión: **files**



```
1 services:
2   app:
3     image: mvarelasanclemente/balanceosessions
4     container_name: balanceo01
5     ports:
6       - "9003:80"
7     environment:
8       APP_NAME: cvd_01
9       SESSION_DRIVER: files
10      REDIS_URL:
```

1. .
2. Para e elimina os contedores.

Explicación Crearemos un servidor Redis gratuíto para poder probar a nosa aplicación almacenando as sesións neste caso en Redis.

1. Agora vai a páxina: `https://upstash.com/`

e crea unha conta de usuario coa conta de correo do IES San Clemente.

Crea unha nova base de datos Redis en `https://upstash.com/`

gratuíta e indica un servidor en Europa.

Anota das credenciais que che dan o nome de dominio e o *token*. Con estes datos debes construír unha URL deste xeito: `tls://dominio:6379?auth=token`

.

Modifica o ficheiro `docker-compose.yaml`

para que utilice o servidor Redis. A URL é a indicada no paso anterior.

Executa o `docker compose up`

e comproba que podes acceder a URL: `localhost:9003`. Actualiza varias veces para ver que o contador de veces que accediches a URL vai aumentando. **Entrega capturas** nas que se vexa como aumenta o contador e o ficheiro `docker-compose.yaml`



Benvido!

Aplicación cvd_01

Accediches a esta páxina **6** veces nesta sesión.

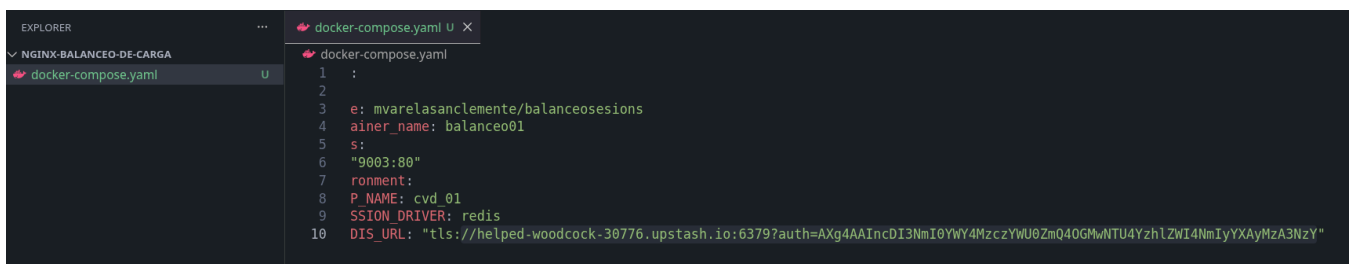
Método de sesión: **redis**

Benvido!

Aplicación cvd_01

Accediches a esta páxina 9 veces nesta sesión.

Método de sesión: **redis**



```
1  :
2
3  e: mwarelasanclemente/balanceosessions
4  ainer_name: balanceo01
5  s:
6  "9003:80"
7  ronment:
8  P_NAME: cvd_01
9  SESSION_DRIVER: redis
10 DIS_URL: "tls://helped-woodcock-30776.upstash.io:6379?auth=AXg4AAIncDI3NmI0YWY4MzczYWU0ZmQ0GhWNTU4YzhLZWl4NmIyYXZlMzA3NzY"
```

1. .

Explicación Agora modificaremos a nosa posta en produción utilizando escalado horizontal: en lugar de un único contedor despregando a aplicación teremos dous. Deixaremos o despregue para que as sesións se almacenen en ficheiros.

1. Modifica o `docker-compose.yaml`

para que agora se volvan utilizar ficheiros para as sesións.

Duplica agora o servizo que tiñamos por outro totalmente igual pero modificando os seguintes datos:

- Nome do contedor: `balanceo02`

Mapea o porto 80 no 9004.

A variable de contorno `APP_NAME`

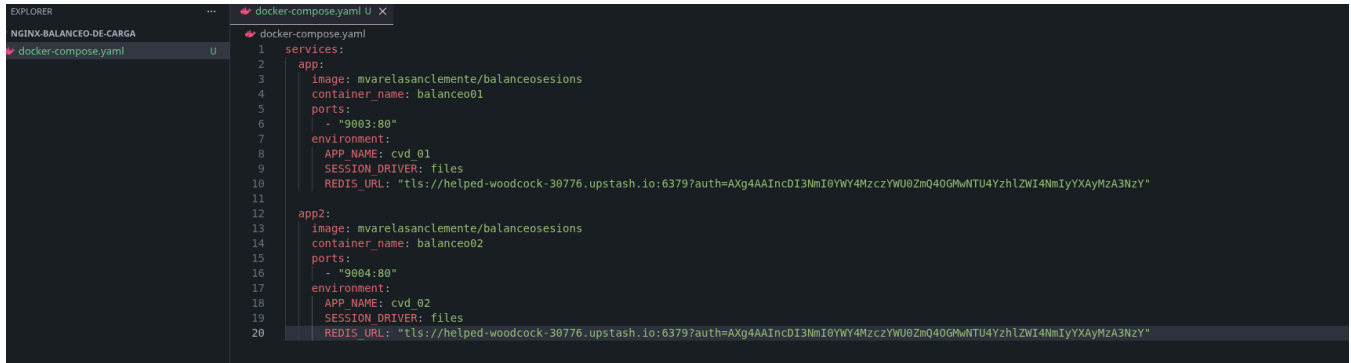
debe de recibir o valor de `iniciais_02`

- .

Executa o `docker compose up`

e comproba que funciona o acceso mediante `localhost:9003` e `localhost:9004`. **Entrega capturas** do ficheiro `docker-compose.yaml`

.



Realiza un `commit`

1. e un `push` ao repositorio.

Exercicio 2: Balanceo de carga

Explicación I mos configurar para que o noso proxy inverso faga balanceo de carga entre os dous contedores que despregan a nosa aplicación. Neste caso os dous contedores están no mesmo servidor, pero podería realizarse para que cada contedor se despregara en servidores diferentes.

1. No servidor de produción de Docker, clona o repositorio `T05.04 Nginx:Balanceo de carga`

Pon en marcha os servizos definidos no repositorio.

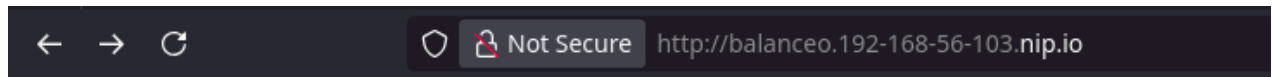
Agora modifica o proxy inverso para realizar balanceo de carga entre os dous servizos levantados no paso anterior. Utiliza o dominio `balanceo.192-168-56-103.nip.io`

. Recorda recargar a configuración. **Entrega capturas** do ficheiro de configuración de Nginx.



Accede agora a `http://balanceo.192-168-56-103.nip.io`

1. nunha ventá privada. **Realiza unha captura** do contido. Actualiza e volve **realizar outra captura**. E así varias veces para ver como o número de accesos a web vai dando saltos entre as dúas aplicación que temos en produción.

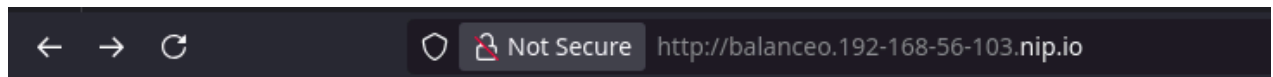


Benvido!

Aplicación cvd_01

Accediches a esta páxina **1** veces nesta sesión.

Método de sesión: **files**

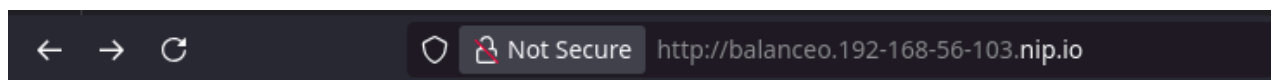


Benvido!

Aplicación cvd_02

Accediches a esta páxina **1** veces nesta sesión.

Método de sesión: **files**



Benvido!

Aplicación cvd_01

Accediches a esta páxina **2** veces nesta sesión.

Método de sesión: **files**

Benvido!

Aplicación cvd_02

Accediches a esta páxina 2 veces nesta sesión.

Método de sesión: **files**

Explicación **A nosa configuración actual non permite escalar horizontalmente. Xa que non é **stateless**.**

Cada un dos contedores almacena as sesións dentro de ficheiros do seu propio contedor. Entón temos información diferente segundo accedamos a un contedor ou a outro. Polo tanto isto non permite realizar escalabilidade horizontal.

A solución como xa vimos é transformar a nosa aplicación en ***stateless***. Almacenaremos as nosas sesións nun servizo externo como pode ser Redis (Existen máis alternativas). Deste xeito conseguiremos poder despregar un número de contedores calquera e manter o funcionamento correcto da aplicación.

1. No servidor de produción, para o despregue e elimina os contedores.
2. No equipo anfitrión modifica o `docker-compose.yaml`

para utilizar o servidor Redis que creamos no exercicio 1. **Entrega capturas** do ficheiro `docker-compose.yaml`

```
GNU nano 7.2                                docker-compose.yaml
services:
  app:
    image: mvarelasanclemente/balanceosesions
    container_name: balanceo01
    ports:
      - "9003:80"
    environment:
      APP_NAME: cvd_01
      SESSION_DRIVER: redis
      REDIS_URL: "tls://helped-woodcock-30776.upstash.io:6379?auth=AXg4AAIncDI3NmI0YWY4MzczYWU0ZmQ4OGMwNTU4Yzh1ZWl4NmIyYXAy"
  app2:
    image: mvarelasanclemente/balanceosesions
    container_name: balanceo02
    ports:
      - "9004:80"
    environment:
      APP_NAME: cvd_02
      SESSION_DRIVER: redis
      REDIS_URL: "tls://helped-woodcock-30776.upstash.io:6379?auth=AXg4AAIncDI3NmI0YWY4MzczYWU0ZmQ4OGMwNTU4Yzh1ZWl4NmIyYXAy"
```

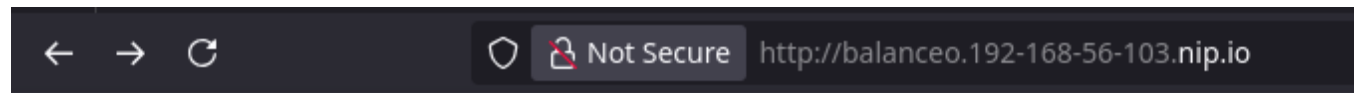
Realiza un `commit`

e un `push` ao repositorio.

No servidor de produción de Docker actualiza o contido do repositorio e lanza os novos servizos.

Accede agora a <http://balanceo.192-168-56-103.nip.io>

nunha nova ventá privada. **Realiza unha captura** do contido. Actualiza e volve **realizar outra captura***. E así varias veces para ver como agora funciona correctamente a aplicación e o número de accesos e secuencial sen importar a aplicación a cal accedamos.

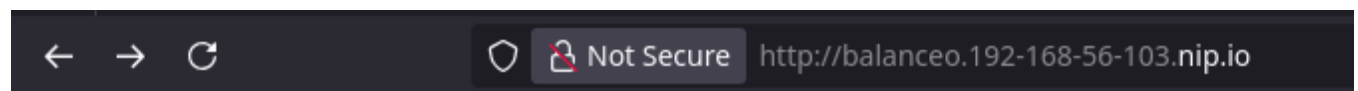


Benvido!

Aplicación cvd_02

Accediches a esta páxina **1** veces nesta sesión.

Método de sesión: **redis**

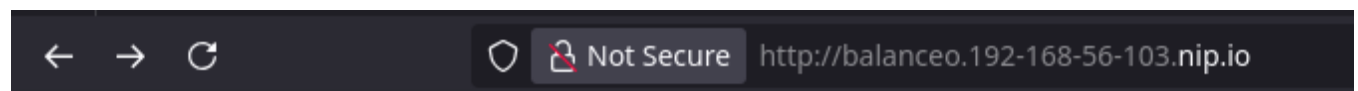


Benvido!

Aplicación cvd_02

Accediches a esta páxina **2** veces nesta sesión.

Método de sesión: **redis**



Benvido!

Aplicación cvd_01

Accediches a esta páxina **3** veces nesta sesión.

Método de sesión: **redis**



Not Secure

http://balanceo.192-168-56-103.nip.io

Benvido!

Aplicación cvd_02

Accediches a esta páxina 4 veces nesta sesión.

Método de sesión: **redis**